PRACTICAL - 01

Practical – 1(a): Create an MVC Application to Perform Basic Arithmetic Operations

Step 1: Create Project

1. Open Visual Studio 2026.
2. Click **Create a new project**.
3. Search MVC.
4. Select **ASP.NET Core Web App (Model-View-Controller)**.
5. Click **Next**.
6. Project Name: ArithmeticMVC
7. Click **Create**.

Step 2: Create Model

1. Open **Models** folder.
2. Right-click → **Add** → **Class**.
3. Name it Calculator.cs.
4. Write:

```
namespace ArithmeticMVC.Models
{
  public class Calculator
  {
    public double Number1 { get; set; }

    public double Number2 { get; set; }

    public string Operation { get; set; } = "";

    public double Result { get; set; }
  }
}
```

Step 3: Controller

1. Open **Controllers**.
2. Open HomeController.cs.
3. Replace the code with:

```
using Microsoft.AspNetCore.Mvc;
using ArithmeticMVC.Models;

namespace ArithmeticMVC.Controllers
{
  public class HomeController : Controller
  {
    [HttpGet]
    public IActionResult Index()
```

```
    {
        return View();
    }

    [HttpPost]
    public IActionResult Index(Calculator calc)
    {
        switch (calc.Operation)
        {
            case "+":
                calc.Result = calc.Number1 + calc.Number2;
                break;

            case "-":
                calc.Result = calc.Number1 - calc.Number2;
                break;

            case "*":
                calc.Result = calc.Number1 * calc.Number2;
                break;

            case "/":
                if (calc.Number2 != 0)
                {
                    calc.Result = calc.Number1 / calc.Number2;
                }
                else
                {
                    ViewBag.Message = "Cannot divide by zero.";
                }
                break;

            default:
                ViewBag.Message = "Please select a valid operation.";
                break;
        }

        return View(calc);
    }
  }
}
```

Step 4: View

1. Open **Views → Home → Index.cshtml**.
2. Delete existing code.
3. Paste:

```
@model ArithmeticMVC.Models.Calculator
```

```
<h2>Arithmetic Operations</h2>
<form method="post">

  Number 1:
  <input asp-for="Number1" />
  <br /><br />

  Number 2:
  <input asp-for="Number2" />
  <br /><br />

  Operation:
  <select asp-for="Operation">
     <option value="+">+</option>
     <option value="-">-</option>
     <option value="*">*</option>
     <option value="/">/</option>
  </select>

  <br /><br />

  <input type="submit" value="Calculate" />

</form>

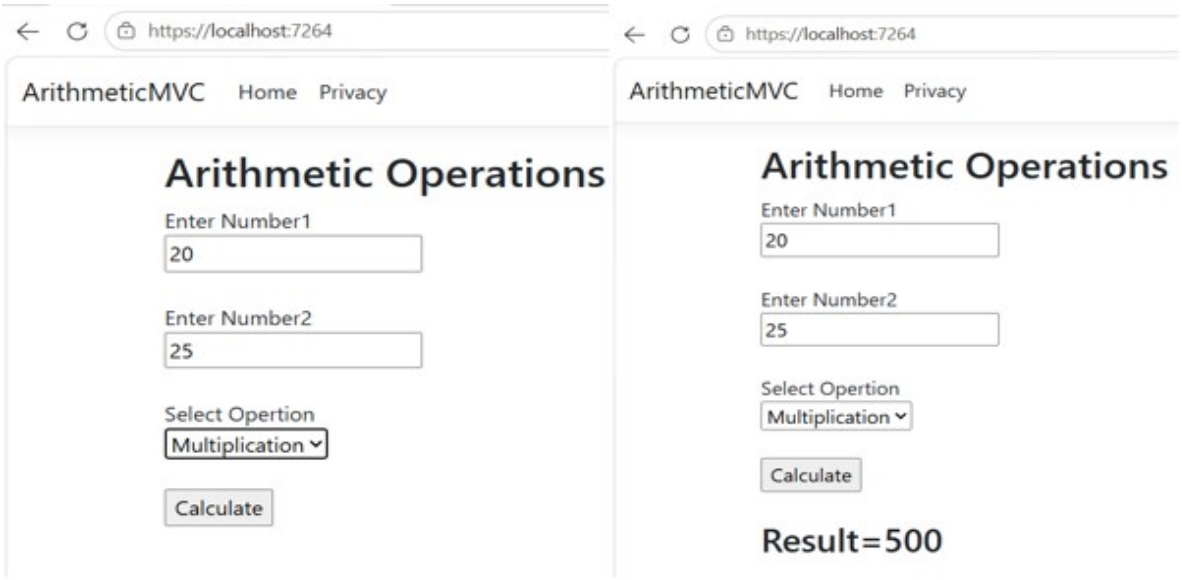
@if (ViewBag.Message != null)
{
  <p>@ViewBag.Message</p>
}

@if (Model != null)
{
  <h3>Result = @Model.Result</h3>
}
```

Step 5: Run

1. Save all files.
2. Press **Ctrl + Shift + B**.
3. Press **Ctrl + F5**.

OUTPUT


https://localhost:7264
ArithmeticMVC
Home
Privacy
Arithmetic Operations
Enter Number1
20
Enter Number2
25
Select Opertion
Multiplication
Calculate
https://localhost:7264
ArithmeticMVC
Home
Privacy
Arithmetic Operations
Enter Number1
20
Enter Number2
25
Select Opertion
Multiplication
Calculate
Result=500

Practical – 1 (b): Create an MVC application to perform basic Arithmetic Operations

Steps

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Program.cs

```
using System;

class Program

{
  static void Main()

  {
     int n, num = 1;

     Console.Write("Enter rows: ");

     n = Convert.ToInt32(Console.ReadLine());

     for (int i = 1; i <= n; i++)

     {
        for (int j = 1; j <= i; j++)

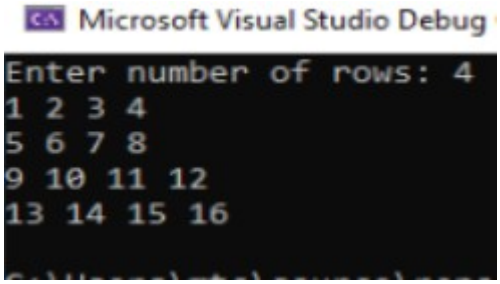
        {
           Console.Write(num + " ");
           num++;
        }

        Console.WriteLine();
     }

     Console.ReadLine();
  }

}
```

OUTPUT


Microsoft Visual Studio Debug
Enter number of rows: 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16

Practical – 1 (c) : Write C# code to arrange the name of cities in sorted order.

Accept name of 10 cities from the user.

Steps:

1. Open Visual Studio 2026
2. Create -> Console App(.NET),Give logical Name to Project as done previously.
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

class Program
{
  static void Main()
  {
     string[] cities = new string[10];

     Console.WriteLine("Enter 10 city names:");

     for (int i = 0; i < 10; i++)
     {
        cities[i] = Console.ReadLine();
     }

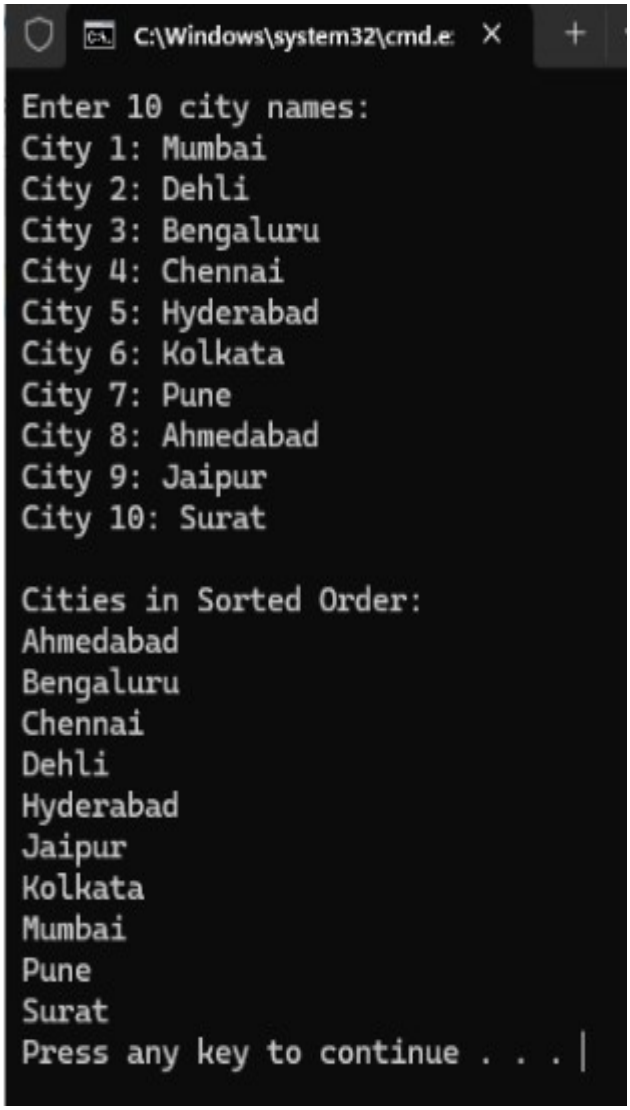
     Array.Sort(cities);

     Console.WriteLine("\nCities in Sorted Order:");

     foreach (string city in cities)
     {
        Console.WriteLine(city);
     }

     Console.ReadLine();
  }
}
```

OUTPUT


C:\Windows\system32\cmd.e
Enter 10 city names:
City 1: Mumbai
City 2: Dehli
City 3: Bengaluru
City 4: Chennai
City 5: Hyderabad
City 6: Kolkata
City 7: Pune
City 8: Ahmedabad
City 9: Jaipur
City 10: Surat
Cities in Sorted Order:
Ahmedabad
Bengaluru
Chennai
Dehli
Hyderabad
Jaipur
Kolkata
Mumbai
Pune
Surat
Press any key to continue . . .

PRACTICAL - 2

Practical – 2 (a) : Write the program to use Function Overloading feature to C#

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete Existing Code
5. Write following Code
6. Run usring Ctrl + F5

Code:

Program.cs

```
using System;

class Addition

{
  public void Add(int a, int b)

  {
     Console.WriteLine("Addition of 2 numbers = " + (a + b));
  }

  public void Add(int a, int b, int c)

  {
     Console.WriteLine("Addition of 3 numbers = " + (a + b + c));

  }

}

class Program

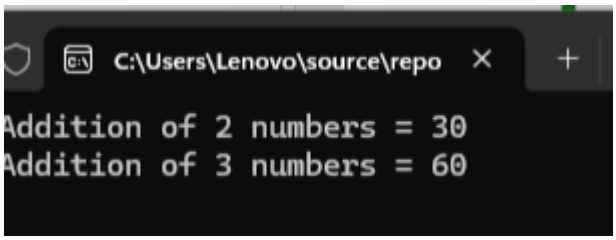
{
  static void Main()

  {
     Addition obj = new Addition();

     obj.Add(10, 20);
     obj.Add(10, 20, 30);
```

```
        Console.ReadLine();
    }
}
```

OUTPUT



Practical – 2 (b) : Write the program to use Inheritance (all types) feature of C#.

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

class Animal

{
  public void Eat()

  {
     Console.WriteLine("Animal can Eat");

  }

}

class Dog : Animal

{
  public void Bark()

  {
     Console.WriteLine("Dog can Bark");

  }

}

class Puppy : Dog

{
  public void Weep()
```

```
  {
    Console.WriteLine("Puppy can Weep");
  }

}

class Cat : Animal

{
  public void Meow()

  {
    Console.WriteLine("Cat can Meow");

  }

}

class Program

{
  static void Main()

  {
    Puppy p = new Puppy();p.Eat();
    p.Bark();
    p.Weep();

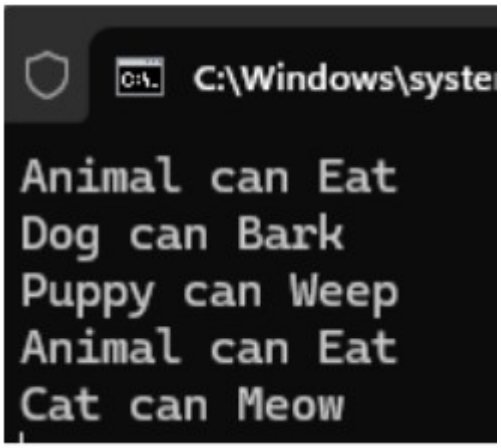
    Cat c = new Cat();

    c.Eat();
    c.Meow();

    Console.ReadLine();
  }

}
```

OUTPUT


C:\Windows\syste
Animal can Eat
Dog can Bark
Puppy can Weep
Animal can Eat
Cat can Meow

Practical – 2 (c) : Write the program to use Constructor overloading feature of C#.

Steps:

7. Open Visual Studio
8. Create à Console App(.net), Give logical Name to Project
9. Open Program.cs
10. Delete existing code
11. Write following code
12. Run using Ctrl + F5

Code:

Program.cs

```
using System;

class Student

{
  public Student()

  {
    Console.WriteLine("Default Constructor Called");
  }

  public Student(string name)

  {
    Console.WriteLine("Student Name: " + name);
  }

  public Student(string name, int age)

  {
    Console.WriteLine("Student Name: " + name);
    Console.WriteLine("Student Age: " + age);

  }

}

class Program

{
  static void Main()
```

```
  {
    Student s1 = new Student();

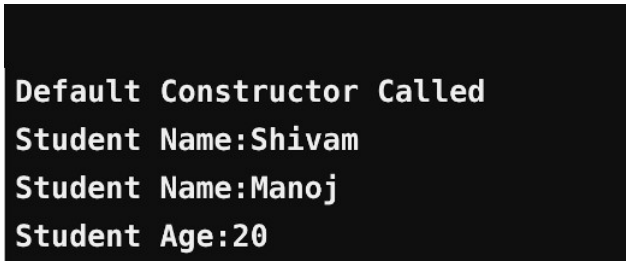
    Student s2 = new Student("Shivam");

    Student s3 = new Student("Manoj", 20);

    Console.ReadLine();
  }

}
```

OUTPUT



Practical – 2 (d) : Write the program to use Inheritance feature of C#.

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

interface IShape

{
  void Area();

}

class Square : IShape

{
  int side = 5;

  public void Area()

  {
    Console.WriteLine("Area of Square = " + (side * side));

  }

}

class Program

{
  static void Main()

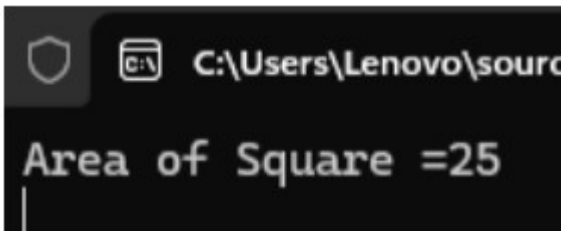
  {
    Square s = new Square();

    s.Area();

    Console.ReadLine();
```

```
    }
}
```

OUTPUT



Practical – 2 (e) : Write the program to use Delegates and events feature of C#.

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

public delegate void Message();

class Test

{
  public event Message Notify;

  public void Show()

  {
    Console.WriteLine("Method inside Test class");

    Notify?.Invoke();
  }

}

class Program

{
  public static void Display()

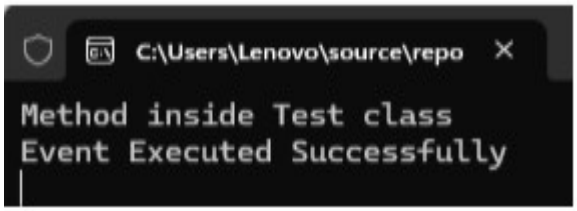
  {
    Console.WriteLine("Event Executed Successfully");
  }

  static void Main()

  {
    Test t = new Test();
```

```
        t.Notify += Display;
        t.Show();
 Console.ReadLine();
    }
}
```

OUTPUT



Practical – 2 (f): Write the program to use Exception handling feature of C#.

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

class Program

{
  static void Main()

  {
     int a, b, result;

     try

     {
        Console.WriteLine("Enter First Number:");

        a = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Enter Second Number:");

        b = Convert.ToInt32(Console.ReadLine());

        result = a / b;

        Console.WriteLine("Result = " + result);

     }

     catch (DivideByZeroException)

     {

        Console.WriteLine("Cannot divide by zero");
```

```
        }
        catch (FormatException)

        {

            Console.WriteLine("Invalid Input");

        }

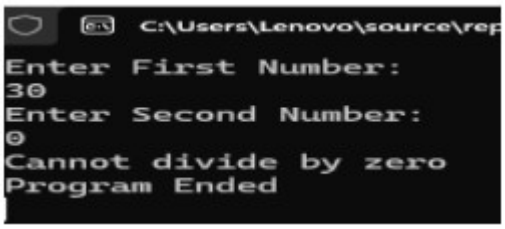
        finally

        {
            Console.WriteLine("Program Ended");

        }

        Console.ReadLine();
    }
}
```

OUTPUT



PRACTICAL - 3

Practical – 3 (a): Create methods add(),multiply(),subtract(),divide() with suitable parameters and call these methods using concept of C# delegate

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

public delegate void Calculation(int a, int b);

class Arithmetic

{
  public void Add(int a, int b)

  {
    Console.WriteLine("Addition = " + (a + b));
  }

  public void Subtract(int a, int b)

  {
    Console.WriteLine("Subtraction = " + (a - b));
  }

  public void Multiply(int a, int b)

  {
    Console.WriteLine("Multiplication = " + (a * b));
  }

  public void Divide(int a, int b)

  {
    Console.WriteLine("Division = " + (a / b));
  }
}
```

```
class Program

{
  static void Main()

  {
     Arithmetic obj = new Arithmetic();

     Calculation c;

     c = obj.Add;
     c(20, 10);

     c = obj.Subtract;
     c(20, 10);

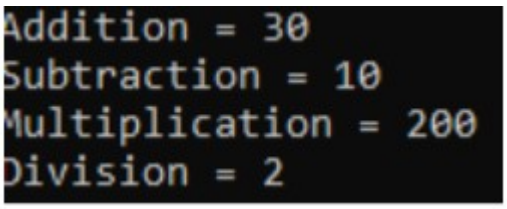
     c = obj.Multiply;
     c(20, 10);

     c = obj.Divide;
     c(20, 10);

     Console.ReadLine();
  }

}
```

OUTPUT



Practical – 3 (b): Write a program using multiple delegate.

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

public delegate void Demo();

class Test

{
  public void Show()

  {
     Console.WriteLine("Show Method Called");
  }

public void Display()

  {

     Console.WriteLine("Display Method Called");

  }
  public void Print()

  {
     Console.WriteLine("Print Method Called");

  }

}

class Program

{
  static void Main()
```

```
    {
        Test t = new Test();

        Demo d;

        d = t.Show;

        d += t.Display;

        d += t.Print;

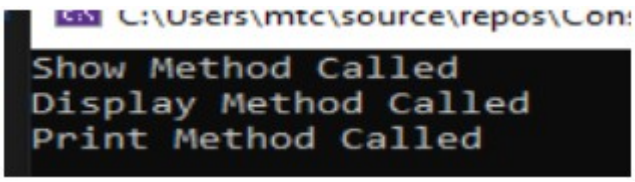
        d();

        Console.ReadLine();

    }

}
```

OUTPUT



Practical – 3 (c): Create a class Bank Account with AccountNumber and Balance.Implement property for Balance with validation(Balance cannot be negative).

Steps:

1. Open Visual Studio
2. Create à Console App(.net), Give logical Name to Project
3. Open Program.cs
4. Delete existing code
5. Write following code
6. Run using Ctrl + F5

Code:

Program.cs

```
using System;

class BankAccount
{
  public int AccountNumber { get; set; }

  private double balance;

  public double Balance
  {
     get
     {
        return balance;
     }

     set
     {
        if (value >= 0)
        {
           balance = value;
        }
        else
        {
           Console.WriteLine("Balance cannot be negative.");
        }
     }
  }
}

class Program
{
```

```
  static void Main()
  {
     BankAccount account = new BankAccount();

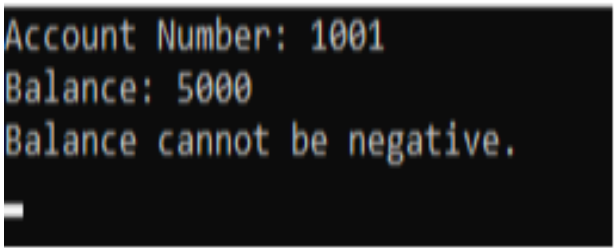
     account.AccountNumber = 1001;
     account.Balance = 5000;

     Console.WriteLine("Account Number: " + account.AccountNumber);
     Console.WriteLine("Balance: " + account.Balance);

     account.Balance = -1000;

     Console.ReadLine();
  }
}
```

OUTPUT



PRACTICAL – 4

Practical – 4 (a): Create an ASP.NET Web Form application using simple server controls

Design a Student Registration Form using the following ASP.NET controls:

i) **Label** and **TextBox** to accept Student Name

ii) **RadioButton** to select Gender (Male/Female)

iii) **DropDownList** to select Course (BSc IT, BSc CS, BCA)

iv) **CheckBox** for accepting Terms & Conditions

v) **Calendar** control to select Date

vi) **Button** control to submit the form

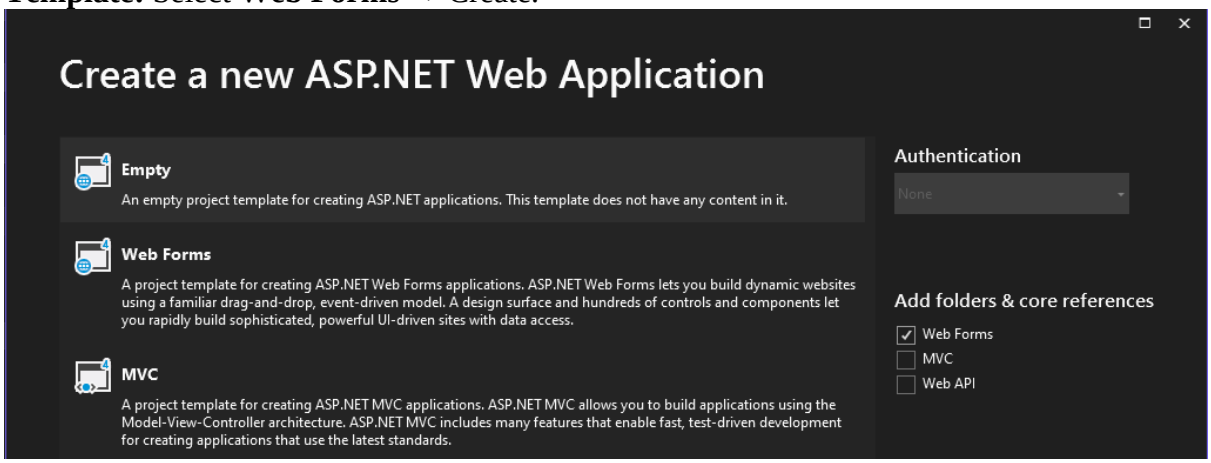
vii) **Label** control to display the output

Step 1 to 6: Project Setup

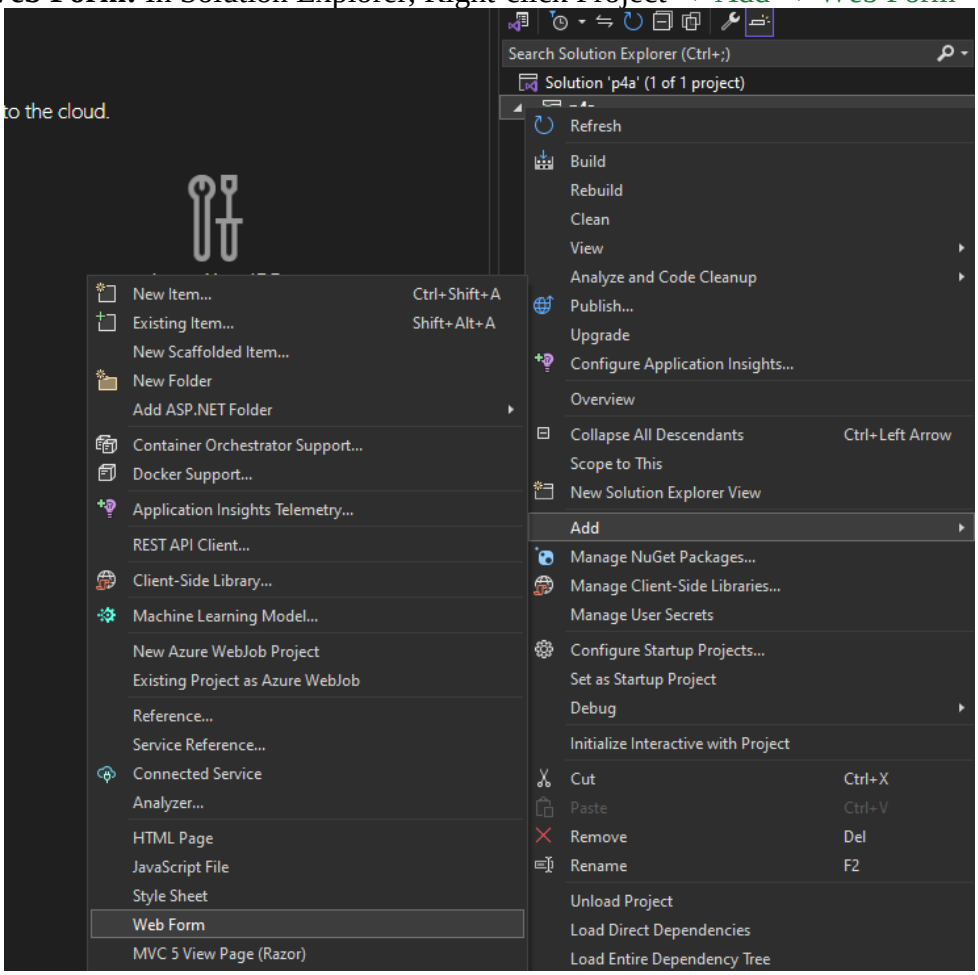
1. **Open Visual Studio** → Create New Project.
2. **Select:** ASP.NET Web Application (.NET Framework) → Next.



3. **Project Name:** StudentRegistration → Create.
4. **Template:** Select **Web Forms** → Create.

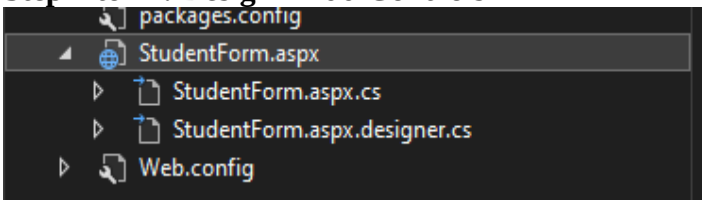


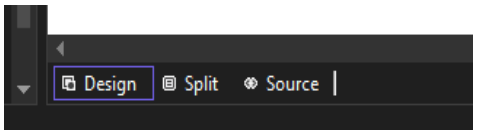
5. **Add Web Form:** In Solution Explorer, Right-click Project → Add → Web Form → Name:



StudentForm.aspx.

6. **Step 7 to 17: Design - Add Controls**





(A) Student Name

- Label → Text = Student Name
- TextBox → ID = txtName

(B) Gender

- Label → Text = Gender
- RadioButton → ID = rbMale
- Text = Male
- GroupName = Gender
- RadioButton → ID = rbFemale
- Text = Female

- GroupName = Gender

(C) Course

- Label → Text = Course
- DropDownList → ID = ddlCourse
- Add Items:
  - BSc IT
  - BSc CS
  - BCA

(D) Terms & Conditions

- CheckBox → ID = chkTerms
- Text = Accept Terms & Conditions

(E) Date

- Calendar → ID = calDate

(F) Submit Button

- Button → ID = btnSubmit
- Text = Submit

(G) Output

- Label → ID = lblOutput
- Leave Text empty.

Step 18 & 19: C# Code - btnSubmit_Click Event

Double-click the Submit Button and paste the following code:

```
protected void btnSubmit_Click(object sender, EventArgs e)
{
  string gender = "";
  if (rbMale.Checked)
  {
     gender = "Male";
  }
  else if (rbFemale.Checked)
  {
     gender = "Female";
```

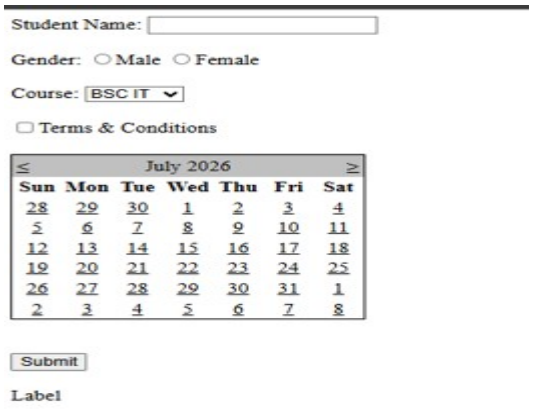
```
    }

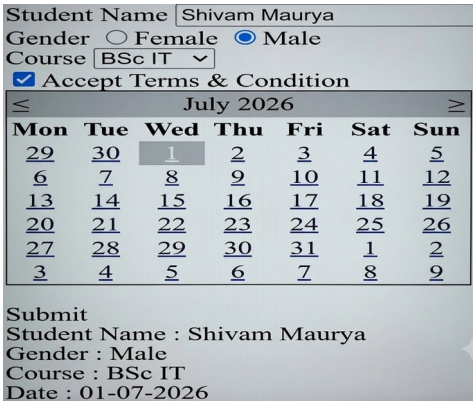
    if (chkTerms.Checked)
    {
        lblOutput.Text = "Student Name: " + txtName.Text +
                         "<br/>Gender: " + gender +
                         "<br/>Course: " + ddlCourse.SelectedValue +
                         "<br/>Date: " + calDate.SelectedDate.ToShortDateString();
    }
    else
    {
        lblOutput.Text = "Please accept Terms & Conditions";
    }
}
```

Step 20 & 21: Run the Application

Press Ctrl + F5 or click **IIS Express**.

OUTPUT





Practical - 4 (b): Create a Registration form to demonstrate use of various Validation Controls

Step 1: Create Project

1. Open Visual Studio 2026
2. Create àASP.NET Web Application(.NET Framework)
3. Select:
    - Empty
    - Click Web Forms
4. Click Create

Step 2: Add Web Form

1. Right-click Project
2. Select Add à Web Form
3. Name: Registration.aspx

Step 3: Open Design View

Open: Registration.aspx

Click: Design

Step 4: Add Controls in Design View

Add the following controls from the **Toolbox**:

1. Add a **Label** with Text: Name:
    - Add a **TextBox**
    - ID: txtName
2. Add a **Label** with Text: Email:
    - Add a **TextBox**
    - ID: txtEmail
3. Add a **Label** with Text: Age:
    - Add a **TextBox**
    - ID: txtAge
4. Add a **Label** with Text: Password:
    - Add a **TextBox**
    - ID: txtPassword
5. Add a **Label** with Text: Confirm Password:
    - Add a **TextBox**
    - ID: txtConfirmPassword
6. Add a **Button**
    - Text: Register
    - ID: btnSubmit

7. Add a **Label**
    - Text: Output
    - ID: lblOutput

Step 5: Add Validation Controls

Add the following validators from **Toolbox → Validation Controls**.

1. RequiredFieldValidator

Add a **RequiredFieldValidator** for the **Name TextBox**.

- **ControlToValidate:** txtName
- **ErrorMessage:** Name Required

2. RegularExpressionValidator

Add a **RegularExpressionValidator** for the **Email TextBox**.

- **ControlToValidate:** txtEmail
- **ErrorMessage:** Invalid Email
- **ValidationExpression:** \w+([-+.']\w+)@\w+([-.]\w+)\.\w+([-.]\w+)

3. RangeValidator

Add a **RangeValidator** for the **Age TextBox**.

- **ControlToValidate:** txtAge
- **MinimumValue:** 18
- **MaximumValue:** 60
- **Type:** Integer
- **ErrorMessage:** Age must be between 18 and 60

4. CompareValidator

Add a **CompareValidator** for the **Confirm Password TextBox**.

- **ControlToValidate:** txtConfirmPassword
- **ControlToCompare:** txtPassword
- **ErrorMessage:** Password does not match

Web.config

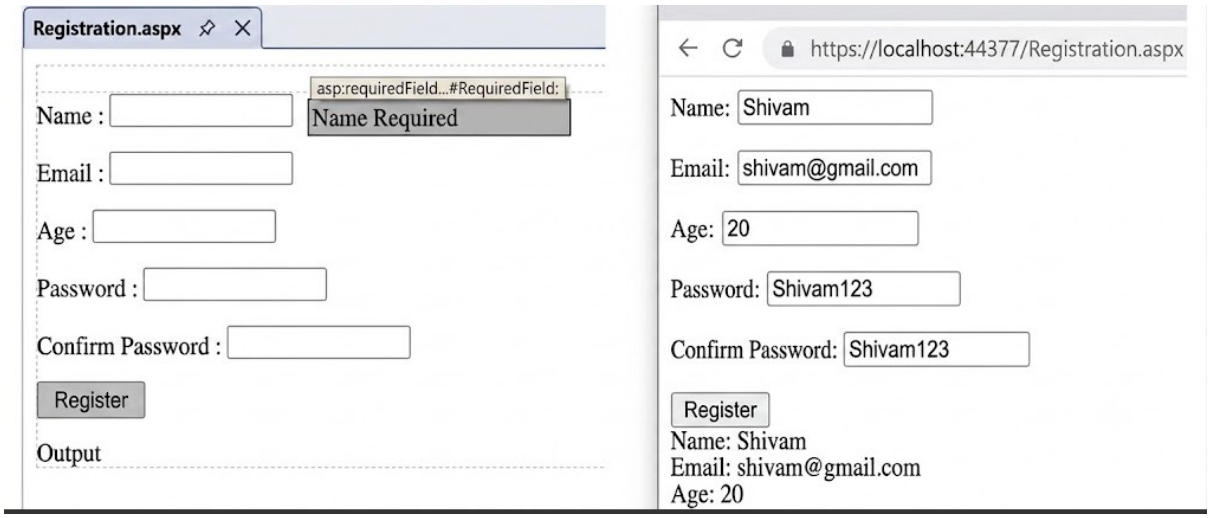
```
<appSettings>
  <add key="ValidationSettings:UnobtrusiveValidationMode"
      value="None"/>
</appSettings>
```

Registration.aspx.cs

```
protected void Page_Load(object sender, EventArgs e)
{
  ValidationSettings.UnobtrusiveValidationMode =
     UnobtrusiveValidationMode.None;
}
protected void btnSubmit_Click(object sender, EventArgs e)
{
  if (Page.IsValid)
  {
     lblOutput.Text = "Registration Successful";
  }
}
```

Save → **Ctrl + Shift + B → Ctrl + F5**

OUTPUT



Practical – 4 (c): Create Web Form to demonstrate use of Adrotator Control.

Step 1: Create Project

1. Open Visual Studio 2022
2. Create à ASP.NET Web Application(.NET Framework)
3. Select:
    - Empty
    - Check Web Forms
4. Click Create

Step 2: Add Web Form

Right-click Project à

Add à Web Form

Name:

AdRotatorDemo.aspx

AdRotatorDemo.aspx

Step 3: Add XML File

Right-click Project à

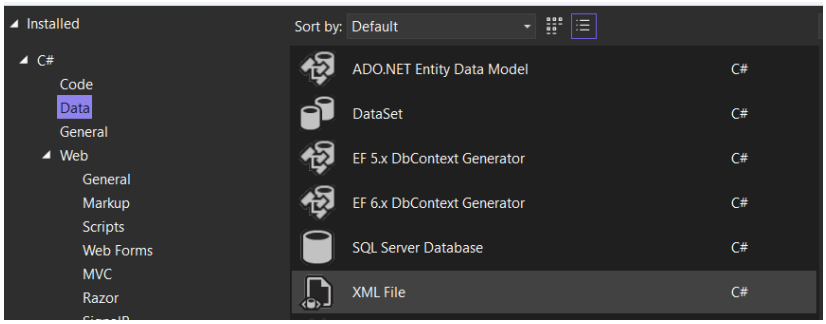
Add à New item

Select:

XML File

Name:

Ads.xml



Step 4: Write XML Code

Open Ads.xml and add the following code:

```
<Advertisements>
    <Ad>
        <ImageUrl>Images/ad1.jpg</ImageUrl>
        <NavigateUrl>https://www.google.com</NavigateUrl>
        <AlternateText>Google Ad</AlternateText>
        <Impressions>50</Impressions>
    </Ad>
    <Ad>
```

<ImageUrl>Images/ad2.jpg</ImageUrl>

<NavigateUrl>https://www.microsoft.com</NavigateUrl>

<AlternateText>Microsoft Ad</AlternateText>

<Impressions>50</Impressions>

</Ad>

</Advertisements>

Step 5: Create Image Folder

Right-click Project à

Add àNew Folder

Name:

Images

Add two images(Download from internet and then add inside folder or copy and paste)

- Ad1.jpg
- Ad2.jpg

Step 6: Open Web Form Design

Open: AdRotatorDemo.aspx From solution Explorer if not opened already

Step 7: Add AdRotator Control

From Toolbox drag:

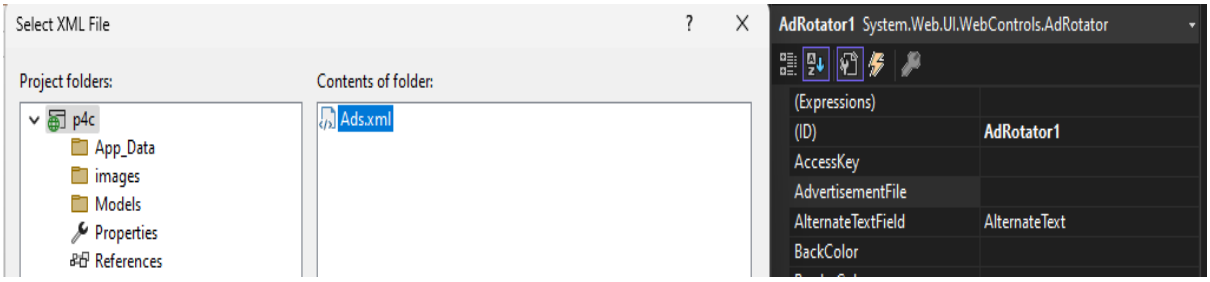
AdRotator

Set Properties:

Property

Value: AdvertisementFile

File: ~/Ads.xml



Step 8: Save All Files

Step 9: Run Application

OUTPUT





PRACTICAL – 5

Practical – 5 (a): Create a web application to demonstrate the use of different types of Cookies.

Step 1: Create Project

1. Open Visual Studio 2022
2. Create à ASP.NET Web Application(.NET Framework)
3. Select:
    - Empty
    - Check Web Forms
4. Click Create

Step 2: Add Web Forms

Right-click Project à

Add àWeb Form

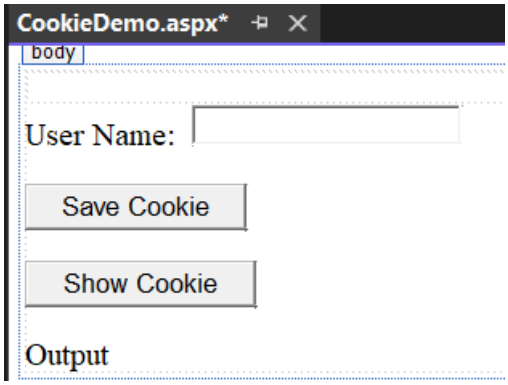
Name:

CookieDemo.aspx

Step 3: Add Controls

Open CookieDemo.aspx in Design View.

Add following controls:



Label Text: User Name

Control Type: TextBox
ID: txtName

Label Text: Save Cookie

Control Type: Button
ID: btnSave

Label Text: Show Cookie

Control Type: Button
ID: btnShow

Label Text: Output

Control Type: Label
ID: lblOutput

Step 4: Double Click Save Cookie Button

Write Code:

```
protected void btnSave_Click(object sender, EventArgs e)
{
        HttpCookie sessionCookie = new HttpCookie("SessionUser");
        sessionCookie.Value = txtName.Text;
        Response.Cookies.Add(sessionCookie);
        HttpCookie persistentCookie = new HttpCookie("PersistentUser");
        persistentCookie.Value = txtName.Text;
        persistentCookie.Expires = DateTime.Now.AddDays(7);
        Response.Cookies.Add(persistentCookie);
        lblOutput.Text = "Cookies Saved Successfully.";
}
```

Step 5: Double Click Show Cookie Button

Write Code:

```
protected void btnShow_Click(object sender, EventArgs e)
{
   string sessionData = Request.Cookies["SessionUser"].Value;
   string persistentData = Request.Cookies["PersistentUser"].Value;
   lblOutput.Text = "Session Cookie: "+sessionData +
            "<br/>Persistent Cookie: "+persistentData;
}
```

Step 6: Save Project

Step 7: Run Application

OUTPUT

User Name

Save Cookie | Show Cookie

[lblOutput]

User Name Shivam Maurya

Save Cookie | Show Cookie

Cookies Saved Successfully

Practical – 5 (b):Create a Product Table in Built in SQL Server of Visual Studio and perform Insert Operation using Entity Framework Core.

Step 1: Open Visual Studio 2022

Create a New Project

Step 2: Create MVC Project

Search: ASP.NET Core Web App (Model – View – Controller)

Click:Next

Step 3: Configure Project

Project Name: ProductEFCore

Framework: .NET 8 or Higher

Click: Create

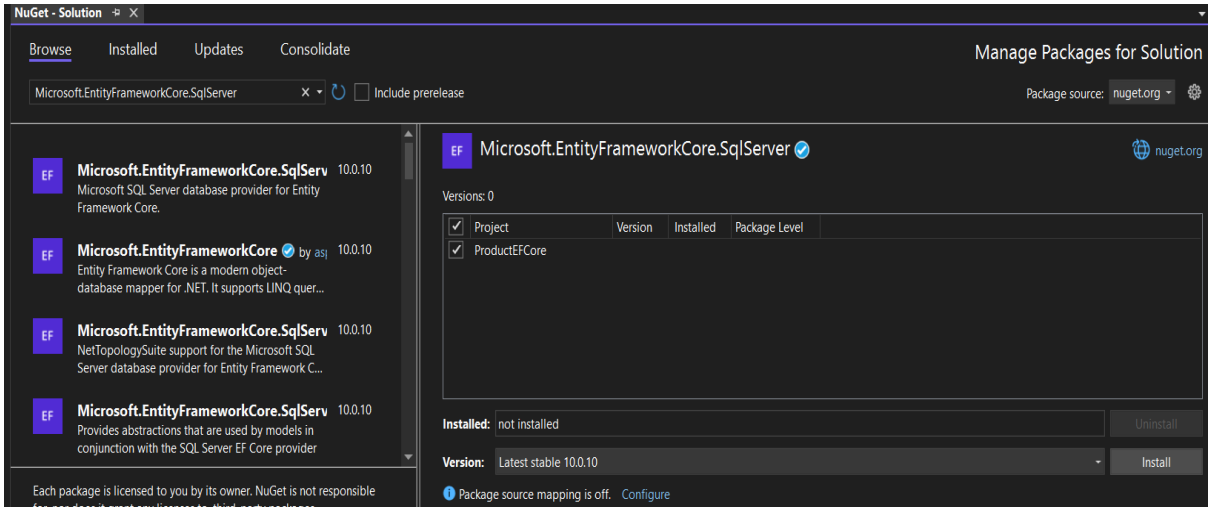
Step 4: Install EF Core Packages

Go to:

Tools àNuGet Package Manager à Manage NuGet Packages for Solution

In Nuget Solution Window select the Browse Tab and in Search Option Enter Following:

Microsoft.EntityFrameworkCore.SqlServer



Select the above name and in right side window Select the Project Name Check Box and then Click on Install.Accepts the terms. Use the same versions for installation

After installing above package do the same for following packages installation.

Microsoft.EntityFrameworkCore

And

Microsoft.EntityFrameworkCore.Tools

And

Microsoft.VisualStudio.Web.CodeGeneration.Design

Step 5: Create Product Model

In Solution Explorer,Right – click :

Model àAddàClass

Name:

Product.cs

Write Code:

```
namespace ProductEFCore.Models
{
   public class Products
   {
      public int Id { get; set; }

      public required string ProductName { get; set; }
      public double Price { get; set; }

   }
}
```

Step 6: Create Data Folder

In Solution Explorer Right – Click Project Name à

Add à New Folder

Name:

Data

Step 7: Create DbContext Class

Right Click Data folder and:

Add à Class

Name:

AppDbContext.cs

Write Code:

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;

namespace ProductEFCore.Data
```

```
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Products> Products { get; set; }
    }
}
```

Step 8: Add Connection String

In Solution Explorer Open: **appsettings.json** file

Add:

```
"ConnectionStrings": {
  "MyConnection": "Server=(localdb)\\
MSSQLLocalDB;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=T
rue"
}
```



Step 9: Register DbContext

From Solution Explorer Open: **Program.cs**

And Add following at the top of file:

using ProductEFCore.Data;

using Microsoft.EntityFrameworkCore;

Before:

var app=builder.Build();

Add:

builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));

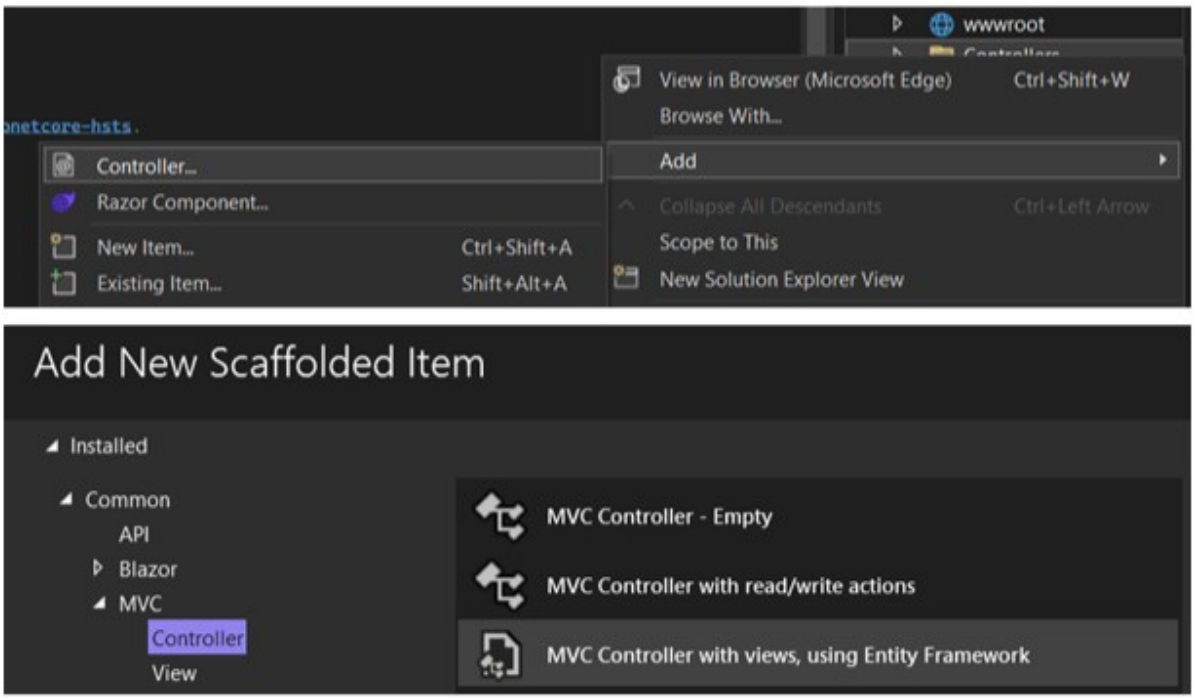
Step 10: Create Product Controller

In Solution Explorer Right-click:

Controllers à Add à Controller

Choose:

MVC Controller with views, using Entity Framework

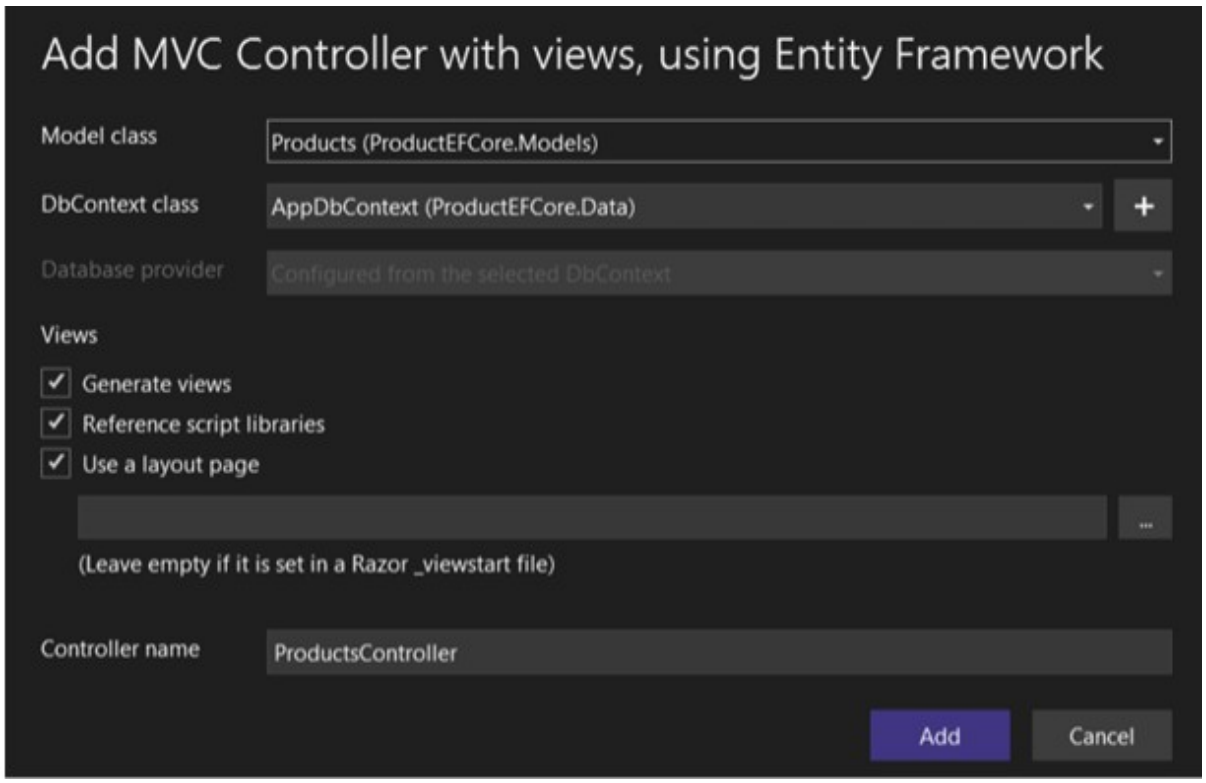


Click: Add

Step 11: Configure Scaffold

Model class: Products

Data context class: AppDbContext



Click: Add

Visual Studio automatically creates:

- Controller
- Insert Page
- Edit Page
- Database operations

Step 12: Create Database using Migration

Go to:

Tools à NuGet Package Manager à Package Manager Console

It opens PM Window at the bottom of IDE, then Run:

Add-Migration InitialCreate

****Note: If the Migration shoes some errors or serious warning then:**

1. **Open the File**

Locate the ProductEFCore.csproj file inside your C:\Users\Administrator\source\repos\ProductEFCore\directory

Open the file with notepad or any good text editor

2. **Paste the Code Fix**

Add following in the same file before the closing </Project> tag.

<ItemGroup>

<NuGetAuditSuppress Include="https;//github.com/advisories/GHSA-g4vj-cjjj-v7hg"/>

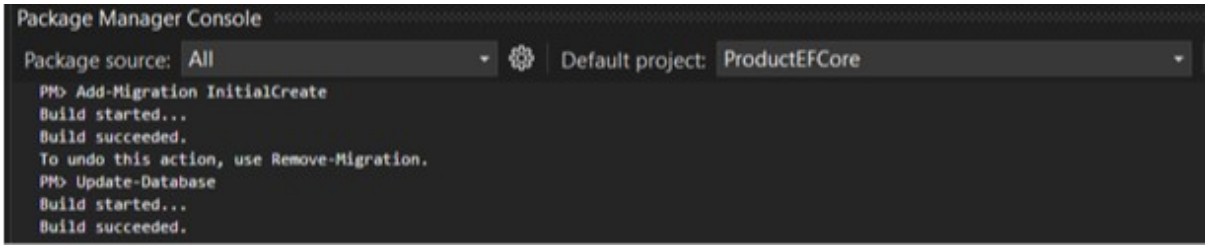
</ItemGroup>

And then build and rebuild the project again and run the Migration command again.

Once Success Then run:

Update-Database

Database and Product table are created automatically in built-in SQL Server.



Step 13: Run Project

Step 14: Open Insert Page

In browser at the end of URL add:

/Products

****Note: If the View Not found error appears in browser,then in Solution Explorer inside View Folder,rename the Product Folder as Products and the Rebuild Project and Run.**

Click:

Create New

Enter:

- Product Name and Price

Click: Create

OUTPUT

Practical – 5 (c): Develop a web applications to perform CRUD Operations(Create,Read,Updata,Delete) on Employee table using EF Core.

Step 1: Create MVC Project

1. Open Visual Studio
2. Create: ASP.NET Core Web App (Model – View – Controller)
3. Project Name: Employee CRUD
4. Framework: .NET 10
5. Click: Create

Step 2: Install EF Core Packages(Same as Previous Example)

Go to:

Tools à NuGet Package Manager à Manage NuGet Packages for Solution

Install:

Microsoft.EntityFrameworkCore.SqlServer
Microsoft.EntityFrameworkCore
Microsoft.EntityFrameworkCore.Tools
Microsoft.VisualStudio.Web.CodeGeneration.Design

Use Same Version:

- 10.0.8 (or Lower but same version for all above packages)

Step 3: Create Employee Model

Right-click:

Models à Add à Class

Name:

Employee.cs

Write code:

```
namespace EmployeeCRUD.Models
{
   public class Employee
   {
      public int Id { get; set; }
      public string Name { get; set; }
      public string Department { get; set; }
      public double Salary { get; set; }
   }
}
```

Step 4: Create Data Folder

Right-click Project à

Add à New Folder

Name:Data

Step 5: Create DbContext Class

Inside Data folder:

Add à Class

Name:

AppDbContext.cs

Write Code:

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;

namespace EmployeeCRUD.Data
{
   public class AppDbContext : DbContext
   {
      public AppDbContext(DbContextOptions<AppDbContext> options)
         : base(options)
      {
      }
      public DbSet<Employee> Employees { get; set; }
   }
}
```

Step 6: Add Connection String

Open:

appsettings.json

Add:

```
"ConnectionStrings": {
  "MyConnection": "Server=(localdb)\\
MSSQLLocalDB;Database=EmployeeDB;Trusted_Connection=True;TrustServerCertificate
=True"
}
```

Step 7: Configure in Program.cs file

Add at top:

```
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Data;
```

Before:

```
var app=builder.Build();
```

Add:

```
builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("MyConnection")));
```

Step 8: Create Employee Controller using Scaffold

Right-click:

Controllers

Select:

Add à New Scaffolded Item

Choose:

MVC Controller with views,using Entity Framework

Click: Add

Step 9: Configure Scaffold

Model Class: Employee

Data Context Class: AppDbContext

Controller Name: EmployeesController

Ensure Checked:

- Generate Views
- Use Layout Page

Click: Add

Visual Studio automatically creates:

- CRUD Views
- Controller
- Database operations

Step 10: Create Database

Open:

Tools à NuGet Package Manager à Package Manager Console

Run:

Add-Migration InitialCreate

****Note: If the Migration shoes some errors or serious warning then:**

1. **Open the File**

   Locate the EmployeeCRUD.csproj file inside your C:\Users\Administrator\source\repos\EmployeeCRUD\directory

   Open the file with notepad or any good text editor

2. **Paste the Code Fix**

   Add following in the same file before the closing </Project> tag.

   <ItemGroup>

   <NuGetAuditSuppress Include=”https;//github.com/advisories/GHSA-g4vj-cjjj-v7hg”/>

   </ItemGroup>

   And then build and rebuild the project again and run the Migration command again.

Then:

Update-Database

Database and Employee table created automatically.

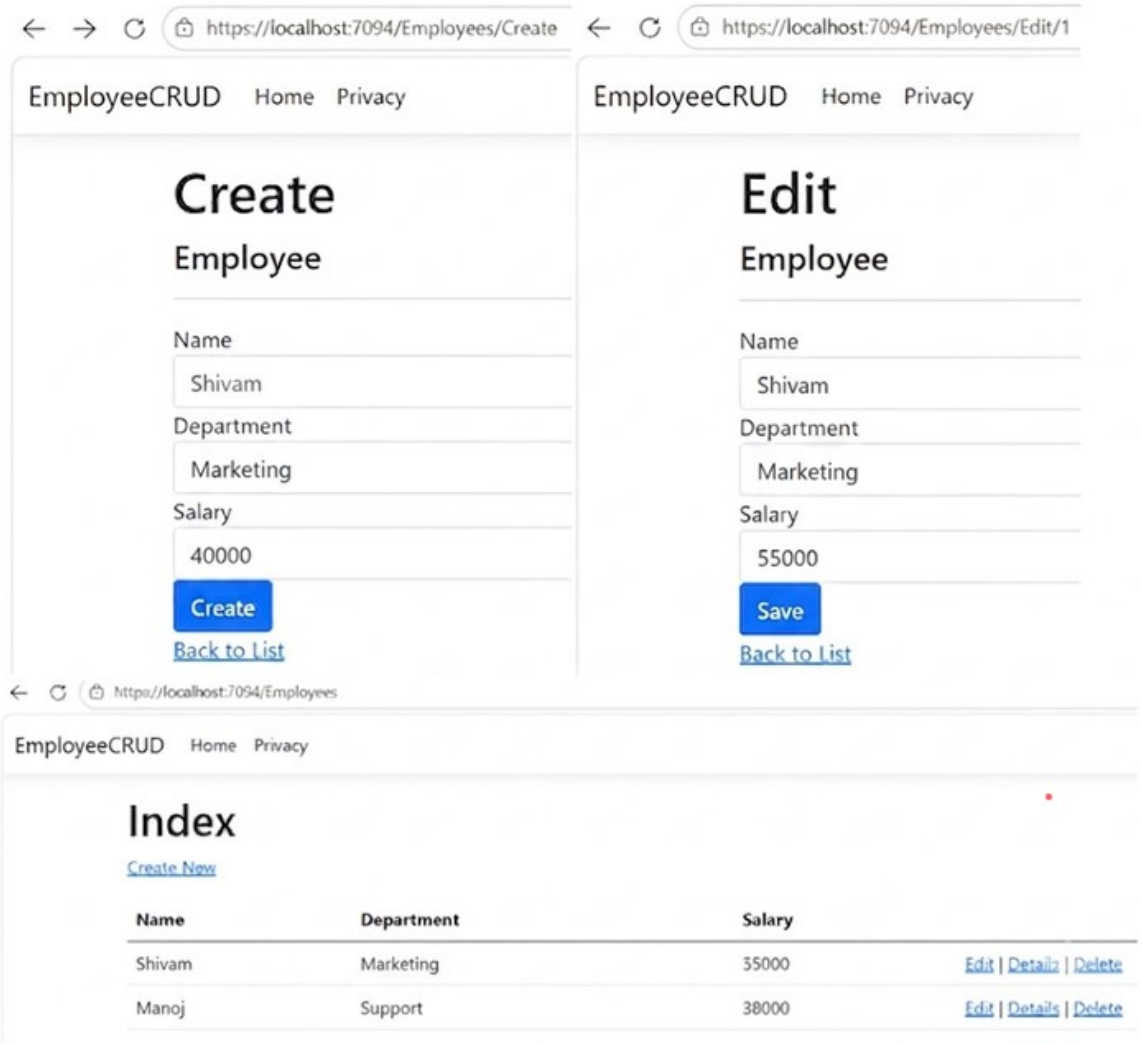
Step 11: Run Application

Open URL and add append following in front of url:

/Employees

****Note: If the View Not found error appears in browser,then in Solution Explorer inside View Folder,rename the Product Folder as Products and the Rebuild Project and Run.**

Output



Practical – 5 (d): Authentication and Authorization in ASP.NET Core.

Step 1: Create MVC Project

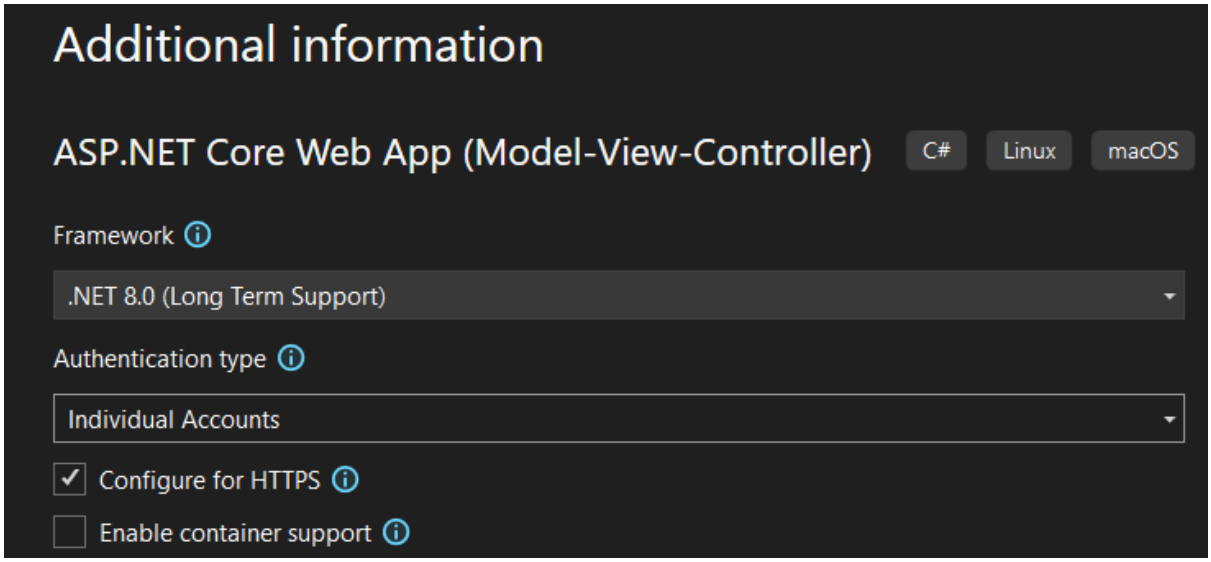
1. Open Visual Studio
2. Create: ASP.NET Core Web App (Model – View – Controller)
3. Project Name: AuthDemo
4. Framework: .NET 08
5. Click Next

Step 2: Select Authentication Type

In Additional Information window:

Choose: Individual Accounts

Click: Create



Visual Studio automatically configure:

- Authentication
- Authorization
- Identity
- Login/Register Pages

Step 3: Build Project

Go to: Build à Rebuild Solution

Step 4: Create Database

Open:

Tools à NuGet Package Manager à Package Manager Console

Run:

Update-Database

Identity tables are automatically created.

Step 5: Run Application

Step 6: Register User

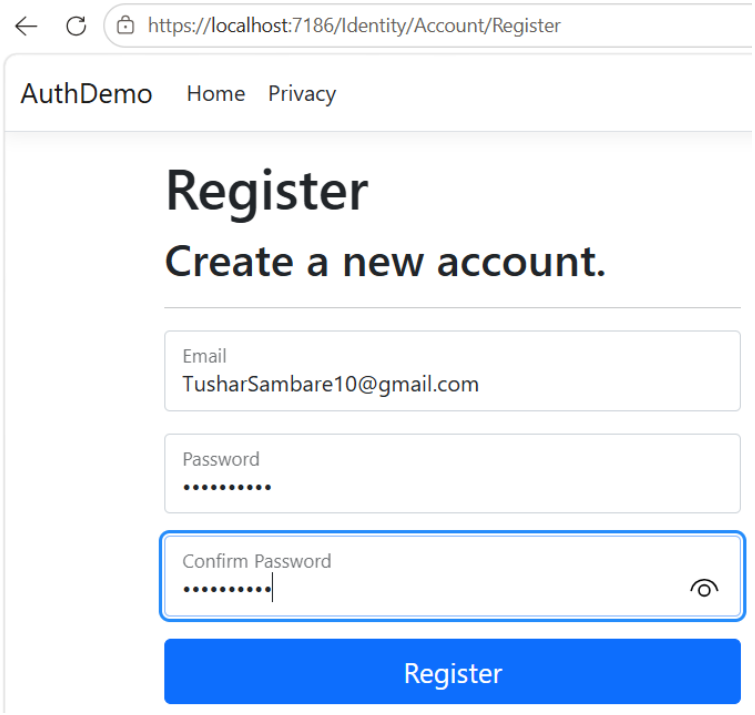
Click Register (At the Top Right Corner of web application)

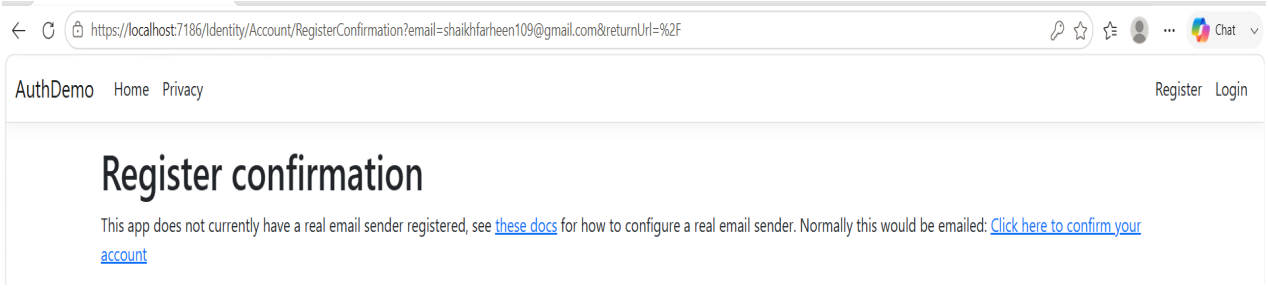
Enter:

- Email
- Password

Click: Register

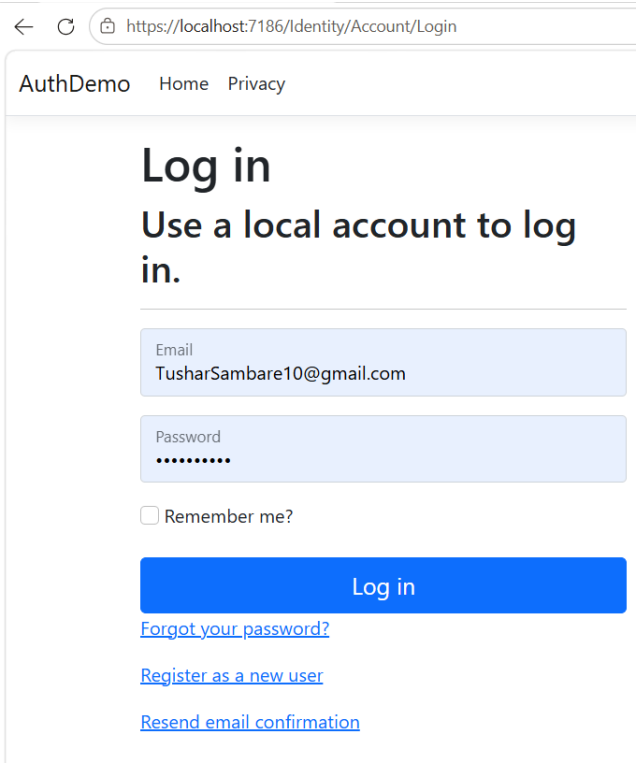
User account created successfully





Step 7: Login

Click: Login (At the Top Right Corner of web application)



Enter credentials.

Step 8: Add Authorized Page

Open:

HomeController.cs

At the top Add:

Using Microsoft.AspNetCore.Authorization;

Add new method:

```
[Authorize]
public IActionResult Secret()
{
  return View();
 }
```

Step 9: Create Secret View

Create folder only if needed: Views/Home

Right Click on Home Folder à Add à View => Select Razor View Empty

Secret.cshtml

Write in empty area at below the prewritten lines:

<h2>Authorized Users Only </h2>

Step 10: Test Authorization

Run with following appending with existing URL:

/Home/Secret

Results:

1. If Previously Logged in then Secret page opens
2. If Previously Not Logged in, then Redirects to Login Page.

OUTPUT

